

Comprehensive Guide: Migrating Boutiquaat Gen App to Amazon EC2

This guide provides a detailed, step-by-step process to successfully migrate your Next.js application (boutiquaat-gen-app) to Amazon EC2 and the AWS ecosystem.

1. Prerequisites

Before starting the migration, ensure you have the following: * An active **AWS Account**. * **AWS CLI** installed and configured on your local machine. * **Git** installed on your local machine and your code pushed to a Git repository (e.g., GitHub, GitLab, or AWS CodeCommit). * A domain name (optional but recommended for production).

2. Infrastructure Setup (AWS EC2)

2.1 Launching an EC2 Instance

1. Log in to the **AWS Management Console**.
2. Navigate to **EC2** and click **Launch Instance**.
3. **Name and Tags:** Give your instance a name, e.g., `boutiquaat-app-server`.
4. **Amazon Machine Image (AMI):** Select **Ubuntu Server 22.04 LTS** (or 24.04 LTS). It's stable and widely used for Node.js apps.
5. **Instance Type:** Select `t3.micro` (Free Tier eligible) for testing, or `t3.medium`/`t3.large` for production depending on your traffic.
6. **Key Pair:** Create a new key pair (RSA, `.pem` format) and download it. **Keep this file secure;** you'll need it to SSH into the server.

Network Settings:

- Create a new Security Group or select an existing one.

Inbound Rules:

- Allow **SSH (Port 22)** from your IP address.
- Allow **HTTP (Port 80)** from anywhere (0.0.0.0/0).

- Allow **HTTPS (Port 443)** from anywhere (0.0.0.0/0).

8. **Storage:** Allocate at least 20 GB of gp3 SSD storage.

9. Click **Launch Instance**.

2.2 Allocate an Elastic IP (Recommended)

By default, EC2 instances get a dynamic public IP that changes upon restart. 1. Go to **Elastic IPs** in the EC2 dashboard. 2. Click **Allocate Elastic IP address** and allocate one. 3. Select the newly created IP, click **Actions > Associate Elastic IP address**, and link it to your `boutiquaat-app-server` instance.

3. Server Configuration

3.1 Connect to Your Instance

Open your terminal and use the key pair you downloaded:

```
chmod 400 your-key-pair.pem ssh -i "your-key-pair.pem" ubuntu@<YOUR_ELASTIC_IP>
```

3.2 Install Dependencies (Node.js, npm, Git, PM2)

Run the following commands on your EC2 instance to set up the environment:

```
# Update package lists sudo apt update && sudo apt upgrade -y # Install Node.js  
(Using NodeSource for latest LTS) curl -fsSL  
https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt-get install -y  
nodejs # Install Git sudo apt install git -y # Install PM2 (Process Manager for  
keeping the app running) sudo npm install -g pm2
```

4. Application Deployment

4.1 Clone the Repository

Generate an SSH key on your EC2 instance and add it to your Git provider, or clone using HTTPS:

```
git clone <YOUR_REPOSITORY_URL> cd boutiquaat-gen-app
```

4.2 Configure Environment Variables

Create a `.env.local` or `.env` file in the project root and add all your production variables (Supabase keys, runninghub keys, etc.).

```
nano .env
```

(Paste your variables, then press `Ctrl+O`, `Enter`, `Ctrl+X` to save and exit).

4.3 Install Application Dependencies

```
npm install
```

4.4 Build the Next.js Application

```
npm run build
```

4.5 Start the Application with PM2

Instead of `npm start`, use PM2 to ensure the app restarts automatically if it crashes or the server reboots.

```
pm2 start npm --name "boutiquaat-app" -- run start
```

To ensure PM2 starts on server boot:

```
pm2 startup # Run the command PM2 outputs, then save the list: pm2 save
```

5. Setting up Nginx as a Reverse Proxy

By default, Next.js runs on port 3000. We use Nginx to route HTTP (port 80) traffic to port 3000.

5.1 Install Nginx

```
sudo apt install nginx -y
```

5.2 Configure Nginx

Create a new configuration file for your app:

```
sudo nano /etc/nginx/sites-available/boutiquaat
```

Paste the following configuration (replace `your_domain_or_IP` with your Elastic IP or domain name):

```
server { listen 80; server_name your_domain_or_IP; location / { proxy_pass  
http://localhost:3000; proxy_http_version 1.1; proxy_set_header Upgrade  
$http_upgrade; proxy_set_header Connection 'upgrade'; proxy_set_header Host  
$host; proxy_cache_bypass $http_upgrade; } }
```

Enable the configuration and restart Nginx:

```
sudo ln -s /etc/nginx/sites-available/boutiquaat /etc/nginx/sites-enabled/ sudo  
nginx -t # Test configuration sudo systemctl restart nginx
```

Your app should now be accessible via your Elastic IP!

6. Securing the Application with SSL (HTTPS)

If you have a domain name pointing to your Elastic IP, secure it using Let's Encrypt.

```
sudo apt install certbot python3-certbot-nginx -y sudo certbot --nginx -d  
your_domain.com
```

Follow the prompts to configure SSL. Certbot will automatically update your Nginx configuration.

7. AWS Ecosystem Enhancements (Optional but Recommended)

To make your application highly scalable and production-ready in AWS:

1. **Amazon RDS:** Move your database to Amazon Relational Database Service (RDS) for automated backups and high availability, rather than hosting it on EC2.
2. **Amazon S3:** Use S3 to store static assets and user uploads instead of the EC2 local storage.
3. **Application Load Balancer (ALB) & Auto Scaling:** If traffic grows, place an ALB in front of multiple EC2 instances running your app, scaling dynamically based on CPU/RAM usage.
4. **AWS Systems Manager:** Use Parameter Store or Secrets Manager to securely manage your `.env` variables.

Summary of Success Checklist

- [x] EC2 Instance launched and accessible via SSH.
- [x] Security groups configured for ports 22, 80, and 443.
- [x] Node.js, Git, and Nginx installed.
- [x] Codebase cloned, environment variables set, and app built successfully.
- [x] App running persistently using PM2.
- [x] Nginx configured as a reverse proxy for port 3000.
- [x] (Optional) SSL certificate applied via Certbot.

Your application is now fully migrated and running robustly on AWS EC2!